CampusConnect (CampSkil) — Viva Voce & Defense Guide

Complete Spring Boot 3 & Full-Stack Project Questions, Answers & Concepts

Java 17 | Spring Boot 3.2.5 | Spring Data JPA | Hibernate | Spring Security | H2 / MySQL | OpenPDF | RESTful APIs

1. ARCHITECTURE & SYSTEM DESIGN

Q1. Can you explain the high-level architecture of CampusConnect?

CampusConnect follows a standard **3-Tier Enterprise Architecture**:

- **Presentation Layer (Client):** Responsive single-page interfaces (HTML5, Tailwind CSS, Vanilla JavaScript) communicating asynchronously via REST APIs (using `fetch()`).
- **Application / Controller Layer:** Built on **Spring Boot 3 (Java 17)** with REST Controllers (`CampusController`) processing client requests and calling business services (`PdfGeneratorService`).
- **Persistence Layer (Database): Spring Data JPA** with Hibernate as the Object-Relational Mapper (ORM), storing data in an embedded **H2 in-memory database** (production-compatible with MySQL).

Q2. Why choose Spring Boot over traditional Spring MVC or Java Servlets?

- **Embedded Server:** Spring Boot includes an embedded Apache Tomcat server, avoiding manual deployment of WAR files to an external server.
- **Starter Dependencies:** Grouped dependencies like `spring-boot-starter-web` and `spring-boot-starter-data-jpa` manage versioning and minimize POM configuration.
- **Automatic Configuration:** Scans the classpath and automatically configures datasources, transaction managers, and security without complex XML.

2. SPRING BOOT CORE ANNOTATIONS

Q3. What does the `@SpringBootApplication` annotation do?

Found on `CampusConnectApplication.java`, it is a meta-annotation that combines:

- `@Configuration`: Flags the class as a configuration source for Spring beans.
- `@EnableAutoConfiguration`: Automatically configures the application context based on classpath dependencies.
- `@ComponentScan`: Automatically scans and registers components, controllers, services, and repositories in the package and its subpackages.

Q4. What is the difference between `@Controller` and `@RestController`?

- `@Controller` is used in traditional MVC web apps where methods return view template names (HTML/JSP) resolved by a view resolver.
- `@RestController` combines `@Controller` and `@ResponseBody`. It indicates that the return value of every method is serialized directly into the HTTP response body as **JSON or XML**.

Q5. How is Dependency Injection implemented in your controller?

We use **Constructor Injection** in `CampusController`. All dependencies (`DriveRepository`, `JobRoleRepository`, `UserRepository`, `RoleRepository`, `PdfGeneratorService`) are marked `final` and passed through the class constructor.
Why? Constructor injection ensures required dependencies cannot be null, produces immutable objects, and makes testing much easier than field injection (`@Autowired`).

3. DATABASE & SPRING DATA JPA

Q6. What is Spring Data JPA, and how does it avoid writing raw SQL?

Spring Data JPA is an abstraction over JPA/Hibernate. By extending `JpaRepository<T, ID>`, Spring dynamically generates proxy implementations at runtime for standard CRUD operations (`findAll`, `findById`, `save`, `delete`) and derived query methods like `findByEmail(String email)` based on method name conventions.

Q7. What is H2 database and how is it configured in this project?

H2 is a fast, lightweight, open-source relational database written in Java that runs entirely embedded in-memory (`jdbc:h2:mem:campus_connect`). It requires zero installation and includes an interactive web console at `/h2-console`. For production, the project can switch to MySQL simply by editing the datasource connection string in `application.properties`.

Q8. What does `spring.jpa.hibernate.ddl-auto=update` do?

Hibernate inspects Java entity classes (`@Entity`) and automatically synchronizes the relational database schema on application launch. If a table or column does not exist, Hibernate creates it; if modified, it adds columns without dropping existing table data.

Q9. How are entity relationships modeled in the application?

- **JobRole ↔ Skill:** Modeled with `@ManyToMany`. A single job role requires multiple skills, and one skill can be associated with multiple roles. It uses an intermediate join table `role_skills`.
- **User ↔ Role:** Modeled with `@ManyToMany` using the `user_roles` join table to assign security roles like `ROLE_STUDENT` and `ROLE_ADMIN`.

4. SECURITY & APPLICATION LIFECYCLE

Q10. What is CORS, and why is `@CrossOrigin(origins = "*")` used?

Cross-Origin Resource Sharing (CORS) is a browser security protocol that prohibits scripts from making requests to a different domain, port, or protocol than the serving page. `@CrossOrigin` instructs the browser to permit cross-origin HTTP requests, allowing external or local frontends to communicate with our Spring Boot API.

Q11. What is `CommandLineRunner`, and why is it used in `DemoDataLoader`?

`CommandLineRunner` is a Spring Boot callback interface. Its `run()` method executes automatically once after the entire Application Context has started up. In `DemoDataLoader`, we use it to seed default drives, skill mappings, and demo user accounts into the database if the tables are empty.

5. CORE FEATURES & FRONTEND INTEGRATION

Q12. How does the dynamic PDF generation work?

In `PdfGeneratorService`, the **OpenPDF (LibrePDF)** library is used. When a user requests `GET /api/drives/{id}/pdf`:

1. The controller retrieves the `OnCampusDrive` record by ID from the database.
2. Sets response headers: `Content-Type: application/pdf` and `Content-Disposition: attachment; filename=drive_details.pdf`.
3. Writes metadata, headers, eligibility criteria, and interview dates directly into the HTTP response `OutputStream`.

Q13. How does the frontend handle authentication and database persistence?

When a student submits the registration form on `test.html`:

- An asynchronous `POST` request is dispatched to `/api/users/register` with user details, saving the student in the database `USERS` table.
- The returned session data is also saved to the browser's `localStorage` (`cc_session`, `cc_display_details`) to ensure fast client-side profile access and state preservation across dashboard navigation.

Pro-Tips for Live Viva & Demonstration

- **Jakarta Namespace:** Remember that Spring Boot 3 uses `jakarta.persistence.*` (Jakarta EE 9/10), not the legacy `javax.persistence.*`.
- **Key API Endpoints:** Know your main endpoints by heart: `/api/drives`, `/api/drives/{id}/pdf`, `/api/skills/roles`, `/api/users/register`, `/api/users/login`, and `/api/health`.
- **Live Database Inspection:** Open `http://localhost:8080/h2-console` during demonstration with JDBC URL `jdbc:h2:mem:campus_connect` to show your live database tables.